

Методические указания преподавателю по контрольной работе

Модуль 3 «Технологии и инструменты обработки данных»

Дисциплина: «Технологии хранения и обработки больших данных»

Общие положения

Параметр	Значение
Модуль	3. Технологии и инструменты обработки данных
Компетенции	BD-4.1 (Уровень П), BD-4.3 (Уровень П), PL-1.1 (Уровень С)
Время выполнения	90 минут
Максимальный балл	35 баллов
Структура	3 блока: тестирование, практическое задание, задание с использованием ИИ

Блок 1. Тестовая часть (14 баллов)

Инструкция: Выберите один правильный ответ из предложенных вариантов.

Вопрос 1. Компетенция BD-4.1. Основы распределенной обработки (1 балл)

Какая технология используется для распределенной обработки больших данных?

- A. Microsoft Excel
- B. Apache Spark
- C. SQL Server Management Studio
- D. Google Sheets

Правильный ответ: В

Обоснование: Apache Spark – это ведущий фреймворк для распределенной обработки больших данных, обеспечивающий параллельные вычисления на кластерах с поддержкой различных парадигм обработки.

Вопрос 2. Компетенция BD-4.1. Пакетная и потоковая обработка (1 балл)

Какая из перечисленных технологий подходит для обработки потоков данных в режиме реального времени?

- A. Apache Kafka
- B. PostgreSQL

C. MySQL

D. TensorFlow

Правильный ответ: A

Обоснование: Apache Kafka является распределенной платформой потоковой передачи данных (streaming platform), позволяющей обрабатывать события в реальном времени с низкой задержкой.

Вопрос 3. Компетенция BD-4.1. Модель MapReduce (1 балл)

Что такое MapReduce в контексте больших данных?

- A. Алгоритм машинного обучения
- B. Модель программирования для обработки больших данных
- C. Метод компрессии данных
- D. Средство визуализации данных

Правильный ответ: B

Обоснование: MapReduce – это фундаментальная модель программирования, разработанная Google, которая реализует распределенную обработку данных через этапы Map (сопоставление) и Reduce (свертка).

Вопрос 4. Компетенция BD-4.1. Преимущества распределенных систем (1 балл)

Какое преимущество имеет распределенное хранилище данных?

- A. Легкость интеграции с любым программным обеспечением
- B. Высокая отказоустойчивость и масштабируемость
- C. Минимальная стоимость внедрения
- D. Возможность работы без доступа к сети

Правильный ответ: B

Обоснование: Распределенные системы обеспечивают отказоустойчивость через репликацию данных и масштабируемость за счет горизонтального добавления узлов.

Вопрос 5. Компетенция BD-4.1. Облачные платформы (1 балл)

Что из перечисленного относится к облачным платформам для работы с большими данными?

- A. Notepad++
- B. Google BigQuery

C. Adobe Photoshop

D. IntelliJ IDEA

Правильный ответ: B

Обоснование: Google BigQuery – это fully-managed серверная платформа для анализа больших данных, основанная на облачной инфраструктуре Google Cloud.

Вопрос 6. Компетенция BD-4.1. ETL-процессы (1 балл)

Как называется процесс преобразования необработанных данных в структурированный вид?

A. MapReduce

B. RDD (Resilient Distributed Dataset)

C. ETL (Extract, Transform, Load)

D. DevOps

Правильный ответ: C

Обоснование: ETL (Extract, Transform, Load) – это классический процесс интеграции данных, включающий извлечение из источников, трансформацию в целевой формат и загрузку в хранилище.

Вопрос 7. Компетенция BD-4.1. Оркестрация конвейеров (1 балл)

Автоматизировать запуск пакетных задач в рамках конвейера обработки больших данных по расписанию можно с помощью:

A. Apache Kafka

B. Apache Airflow

C. Apache Hadoop

D. PostgreSQL

Правильный ответ: B

Обоснование: Apache Airflow предназначен для оркестрации сложных вычислительных конвейеров, позволяя задавать зависимости между задачами и управлять их выполнением по расписанию.

Вопрос 8. Компетенция BD-4.3. Форматы хранения (1 балл)

Формат Parquet считается:

A. полуструктурированным

- В. колоночным (столбцовым)
- С. строковым
- Д. неструктурированным

Правильный ответ: В

Обоснование: Parquet – это колоночный формат хранения данных, оптимизированный для аналитических запросов, обеспечивающий высокую степень сжатия и эффективное чтение отдельных колонок.

Вопрос 9. Компетенция BD-4.1. Потокковая обработка (1 балл)

Выберите технологию потокковой обработки событий в режиме реального времени:

- A. Spark Streaming
- B. Apache Hadoop
- C. MapReduce
- D. Oracle Database

Правильный ответ: А

Обоснование: Spark Streaming – это компонент Apache Spark для обработки потоков данных в режиме реального времени, использующий микро-пакеты для достижения низкой задержки.

Вопрос 10. Компетенция BD-4.3. Прослеживаемость и качество данных (1 балл)

Что из перечисленного относится к задачам управления метаданными в системах больших данных?

- A. Только хранение сырых данных
- B. Отслеживание происхождения данных (Data Lineage) и контроль качества
- C. Исключительно визуализация результатов
- D. Оптимизация SQL-запросов

Правильный ответ: В

Обоснование: Управление метаданными включает отслеживание происхождения данных (Data Lineage) и контроль их качества, что позволяет понимать жизненный цикл данных и обеспечивать их достоверность.

Вопрос 11. Компетенция BD-4.1. Массово-параллельные системы (1 балл)

Какая характеристика соответствует массово-параллельным системам (Massively Parallel Processing, MPP)?

- A. Последовательная обработка данных на одном узле
- B. Распределенная обработка с разделением данных между множеством узлов
- C. Обработка только потоковых данных
- D. Вычисления только в оперативной памяти без хранения

Правильный ответ: B

Обоснование: MPP-системы распределяют данные и вычисления между множеством независимых узлов, каждый из которых имеет собственные процессор, память и хранилище.

Вопрос 12. Компетенция BD-4.1. Инструменты маршрутизации данных (1 балл)

Apache NiFi используется для:

- A. визуализации результатов аналитики
- B. маршрутизации потоков больших данных и построения конвейеров преобразования данных
- C. вычислений в реальном времени
- D. хранения данных на диске

Правильный ответ: B

Обоснование: Apache NiFi – это система для автоматизации потоков данных, обеспечивающая маршрутизацию, трансформацию и доставку данных между системами.

Вопрос 13. Компетенция BD-4.1. Распределенные базы данных (1 балл)

Какая из перечисленных баз данных относится к колоночным (Column-Family) хранилищам?

- A. MongoDB
- B. Cassandra
- C. Redis
- D. Neo4j

Правильный ответ: B

Обоснование: Apache Cassandra – это распределенная база данных с архитектурой column-family, обеспечивающая высокую доступность и масштабируемость без единой точки отказа.

Вопрос 14. Компетенция BD-4.3. Показатели качества данных (1 балл)

Какой из перечисленных показателей относится к характеристикам качества данных (Data Quality)?

- A. Только объем данных
- B. Полнота, точность, своевременность, согласованность
- C. Скорость передачи данных
- D. Сложность запросов

Правильный ответ: B

Обоснование: Качество данных оценивается по нескольким измерениям, включая полноту, точность, своевременность и согласованность.

Блок 2. Практическое задание (16 баллов)

Задание 2.1. Компетенции BD-4.1, PL-1.3. Разработка архитектуры конвейера (6 баллов)

Условие: Компания «Данные-Тех» собирает логи посещений своего веб-сайта. Данные поступают в виде файлов логов (формат JSON) в папку /data/raw/ на сервере. Необходимо:

1. Ежедневно обрабатывать новые файлы логов.
2. Очищать данные от дубликатов и записей с отсутствующими обязательными полями.
3. Трансформировать данные: извлекать user_id, page_url, timestamp, session_duration, referrer.
4. Сохранять обработанные данные в партиционированную таблицу в хранилище.
5. Обеспечивать прослеживаемость данных (Data Lineage).
6. Контролировать качество данных на каждом этапе.

Задание:

A. Опишите архитектуру пайплайна (2 балла)

Изобразите схему конвейера с указанием всех компонентов, потоков данных и технологий.

Ожидаемый ответ: Схема должна включать:

- **Extract:** Чтение JSON-файлов из /data/raw/ (возможно, через FileSensor в Airflow).
 - **Transform:** Очистка (удаление дубликатов, фильтрация NULL-значений) и трансформация (извлечение полей, преобразование типов) на Spark.
 - **Load:** Запись в партиционированную таблицу в ClickHouse или PostgreSQL.
 - **Оркестрация:** DAG в Apache Airflow.
 - **Data Lineage:** OpenMetadata для отслеживания происхождения данных.
 - **DQ:** Проверки на каждом этапе.
-

B. Напишите код графа зависимостей задач для Apache Airflow (2 балла)

Составьте граф зависимостей задач (DAG), который реализует описанный конвейер. Используйте операторы Python или Spark.

Пример правильного ответа:

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.providers.apache.spark.operators.spark_submit import SparkSubmitOperator
from datetime import datetime, timedelta

default_args = {
    'owner': 'data_team',
    'depends_on_past': False,
    'start_date': datetime(2026, 1, 1),
    'email_on_failure': True,
    'email_on_retry': False,
    'retries': 2,
    'retry_delay': timedelta(minutes=5),
}

dag = DAG(
    'web_logs_processing',
    default_args=default_args,
    description='Обработка логов веб-сайта',
    schedule_interval='@daily',
    catchup=False,
    tags=['logs', 'etl'],
)

# Задача: Проверка наличия новых файлов
check_new_files = PythonOperator(
    task_id='check_new_files',
    python_callable=check_files,
    dag=dag,
)

# Задача: Spark-обработка (очистка и трансформация)
spark_process = SparkSubmitOperator(
    task_id='spark_process_logs',
    application='/opt/spark-apps/process_logs.py',
    conn_id='spark_default',
    verbose=True,
    dag=dag,
)

# Задача: Загрузка в хранилище
load_to_warehouse = PythonOperator(
    task_id='load_to_clickhouse',
    python_callable=load_to_clickhouse,
    dag=dag,
)

# Задача: Data Quality проверка
dq_check = PythonOperator(
    task_id='quality_check',
    python_callable=run_dq_checks,
    dag=dag,
)

# Задача: Обновление Data Lineage
update_lineage = PythonOperator(
    task_id='update_lineage',
    python_callable=update_openmetadata,
    dag=dag,
)

# Порядок выполнения
check_new_files >> spark_process >> load_to_warehouse >> dq_check >> update_lineage
```

С. Напишите код Spark-приложения (2 балла)

Реализуйте на PySpark логику очистки и трансформации логов: удаление дубликатов, фильтрация записей без обязательных полей (user_id, timestamp), извлечение необходимых полей в целевой формат.

Пример правильного ответа:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, from_json, when, current_timestamp
from pyspark.sql.types import StructType, StructField, StringType, LongType

spark = SparkSession.builder \
    .appName("LogsProcessing") \
    .config("spark.sql.adaptive.enabled", "true") \
    .config("spark.sql.adaptive.coalescePartitions.enabled", "true") \
    .getOrCreate()

# Схема для чтения JSON-логов
log_schema = StructType([
    StructField("user_id", StringType(), True),
    StructField("page_url", StringType(), True),
    StructField("timestamp", LongType(), True),
    StructField("session_duration", LongType(), True),
    StructField("referrer", StringType(), True),
])

# Чтение сырых данных
df = spark.read \
    .option("header", "true") \
    .schema(log_schema) \
    .json("/data/raw/*.json")

# 1. Удаление дубликатов
df_cleaned = df.dropDuplicates(["user_id", "page_url", "timestamp"])

# 2. Фильтрация записей с обязательными полями
df_filtered = df_cleaned.filter(
    col("user_id").isNotNull() &
    col("timestamp").isNotNull()
)

# 3. Трансформация: преобразование timestamp в дату
df_transformed = df_filtered.withColumn(
    "processed_date",
    from_unixtime(col("timestamp"), "yyyy-MM-dd")
)

# 4. Запись в целевое хранилище (партиционирование по дате)
df_transformed.write \
    .mode("overwrite") \
    .partitionBy("processed_date") \
    .parquet("/data/processed/logs/")

spark.stop()
```

Задание 2.2. Компетенция BD-4.3. Анализ и оптимизация (5 баллов)

Условие: у вас есть Spark-приложение, которое обрабатывает 50 ГБ сырых логов за 4 часа. Время выполнения кажется слишком большим. Проанализируйте возможные причины низкой производительности и предложите способы оптимизации.

Задание: перечислите не менее 4 причин медленной работы и предложите конкретные решения для каждой из них.

Критерии оценки:

- Каждая причина оценивается в 1 балл (всего 4 балла).
- За аргументированное обоснование решений — бонусный 1 балл.

Ожидаемый ответ:

Причина	Решение	Обоснование
1. Неоптимальное количество партиций	Использовать <code>df.repartition(200)</code> или <code>df.coalesce(50)</code> в зависимости от операций	Слишком мало партиций создает узкое место; слишком много – избыточные накладные расходы. Оптимально: 2-4 партиции на ядро CPU
2. Отсутствие кэширования промежуточных данных	Использовать <code>df.cache()</code> или <code>df.persist(StorageLevel.MEMORY_AND_DISK)</code> для данных, используемых несколько раз	Кэширование предотвращает повторное чтение данных из хранилища при каждом действии
3. Неэффективные форматы данных (CSV/JSON)	Использовать колоночные форматы (Parquet/ORC) на этапе чтения и хранения	Parquet обеспечивает лучшее сжатие, пропуск колонок и predicate pushdown, что ускоряет чтение
4. Неправильное использование широких трансформаций	Минимизировать использование <code>groupByKey</code> , заменить на <code>reduceByKey</code> или <code>aggregateByKey</code>	<code>groupByKey</code> вызывает полную перетасовку всех данных, в то время как <code>reduceByKey</code> частично агрегирует на мапперах
5. Избыточное количество файлов при записи	Использовать <code>coalesce(n)</code> перед записью для уменьшения числа файлов	Много маленьких файлов создает нагрузку на NameNode и замедляет последующее чтение
6. Отсутствие настройки <code>spark.sql.shuffle.partitions</code>	Установить <code>spark.sql.shuffle.partitions = 200</code> (по умолчанию 200, может потребоваться больше)	Количество партиций для перетасовки по умолчанию может быть неоптимально для вашего объема данных

Задание 2.3. Компетенция BD-4.3. Контроль качества данных (5 баллов)

Условие: В конвейере преобразования данных на разных этапах могут возникать ошибки, связанные с качеством данных. Вам необходимо реализовать систему мониторинга качества данных.

Задание:

А. Опишите, какие показатели качества данных (DQ) вы будете проверять (2 балла)

Ожидаемый ответ:

Показатель	Описание	Пример проверки
Полнота (Completeness)	Наличие всех обязательных полей в записи	Проверка, что <code>user_id</code> , <code>timestamp</code> не равны NULL

Показатель	Описание	Пример проверки
Уникальность (Uniqueness)	Отсутствие дублирующихся записей	Количество уникальных записей по ключевым полям
Согласованность (Consistency)	Соответствие данных заданным правилам	Проверка, что timestamp не в будущем и не ранее 2000 года
Точность (Accuracy)	Соответствие данных реальным значениям	Проверка, что session_duration > 0
Актуальность (Timeliness)	Данные соответствуют ожидаемому временному окну	Количество записей за последние 24 часа
Допустимые значения (Validity)	Значения находятся в допустимых диапазонах	Проверка формата user_id (UUID)

В. Напишите функцию на Python для автоматической проверки качества данных (3 балла)

Реализуйте функцию, которая принимает таблицу данных и возвращает отчет с метриками качества.

Ожидаемый ответ:

```
import pandas as pd
from datetime import datetime

def dq_report(df, required_columns=None, id_columns=None):
    """
    Генерация отчета о качестве данных.

    Parameters:
    - df: pandas DataFrame
    - required_columns: список обязательных колонок
    - id_columns: список колонок-идентификаторов для проверки уникальности

    Returns:
    - dict с метриками качества
    """
    report = {}
    total_rows = len(df)
    report['total_rows'] = total_rows

    # 1. Проверка полноты (Completeness)
    if required_columns:
        completeness = {}
        for col in required_columns:
            null_count = df[col].isnull().sum()
            completeness[col] = {
                'null_count': null_count,
                'null_percentage': round(null_count / total_rows * 100, 2),
                'is_valid': null_count == 0
            }
        report['completeness'] = completeness

    # 2. Проверка уникальности (Uniqueness)
    if id_columns:
        duplicate_count = df.duplicated(subset=id_columns).sum()
        report['uniqueness'] = {
            'duplicate_count': duplicate_count,
            'duplicate_percentage': round(duplicate_count / total_rows * 100, 2),
            'is_valid': duplicate_count == 0
        }
```

```

# 3. Проверка допустимых значений (Validity) для числовых колонок
numeric_metrics = {}
for col in df.select_dtypes(include=['number']).columns:
    numeric_metrics[col] = {
        'min': df[col].min(),
        'max': df[col].max(),
        'mean': round(df[col].mean(), 2),
        'std': round(df[col].std(), 2),
        'null_count': df[col].isnull().sum()
    }
report['numeric_metrics'] = numeric_metrics

# 4. Проверка на выбросы (Outliers) по IQR
outliers = {}
for col in df.select_dtypes(include=['number']).columns:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outliers[col] = {
        'lower_bound': lower_bound,
        'upper_bound': upper_bound,
        'outliers_count': ((df[col] < lower_bound) | (df[col] > upper_bound)).sum()
    }
report['outliers'] = outliers

# 5. Общий статус качества
is_valid = True
if 'completeness' in report:
    for col, metrics in report['completeness'].items():
        if not metrics['is_valid']:
            is_valid = False
if 'uniqueness' in report:
    if not report['uniqueness']['is_valid']:
        is_valid = False

report['overall_status'] = 'PASS' if is_valid else 'FAIL'
report['generated_at'] = datetime.now().isoformat()

return report

# Пример использования
df = pd.read_csv('processed_logs.csv')
report = dq_report(
    df,
    required_columns=['user_id', 'timestamp', 'page_url'],
    id_columns=['user_id', 'timestamp']
)
print(report['overall_status'])

```

Блок 3. Задания с использованием искусственного интеллекта (5 баллов)

Задание 3.1. Компетенция PL-1.3. Генерация тестовых данных с помощью ИИ (2 балла)

Условие: для тестирования конвейера обработки данных вам необходимо сгенерировать синтетический набор журналов веб-сервера в формате JSON.

Задание:

А. Сформулируйте запрос к генеративному искусственному интеллекту (ChatGPT, YandexGPT или GigaChat) для генерации 1000 записей журналов с полями: user_id, page_url, timestamp, session_duration, referrer. В запросе укажите требования к формату данных, диапазонам значений и структуре JSON. **(1 балл)**

****В.** Получите сгенерированные данные, сохраните их в файл test_logs.json и напишите код на Python для чтения этого файла и вывода первых 5 записей. **(1 балл)**

Задание 3.2. Компетенция PL-1.3. Проверка кода с помощью ИИ (3 балла)

Условие: представьте, что вы написали код для очистки и трансформации журналов, но не уверены в его оптимальности. Используйте генеративный искусственный интеллект для проверки кода.

Задание:

А. Сформулируйте запрос к генеративному искусственному интеллекту для проверки следующего кода. В запросе укажите необходимость выявления:

- потенциальных ошибок;
- проблем с производительностью;
- нарушений лучших практик написания кода. **(1 балл)**

```
from pyspark.sql import functions as F
def clean_data(df):
    df = df.dropDuplicates(["user_id"])
    df = df.filter(F.col("price") < 100000)
    df = df.withColumn("year", F.col("year").cast("int"))
    return df
```

****В.** Получите ответ от искусственного интеллекта, зафиксируйте его в отчете. На основе полученных рекомендаций исправьте код и приложите исправленную версию с комментариями, поясняющими внесенные изменения. **(2 балла)**